

Blazor WebAssembly Model Editing Patterns with Server-Side Validation

Executive Summary

Blazor WebAssembly developers have evolved sophisticated patterns for handling complex model editing with server-side validation, ranging from simple wrapper classes to enterprise-grade architectures. **The most effective approaches combine editor wrapper patterns with dedicated validation containers, leveraging Blazor's built-in EditContext system while maintaining clean separation between editing state and domain models.** [Microsoft Learn](#) [microsoft](#) These patterns enable reusable, type-safe editing across different model types without requiring external state management libraries. [CodeProject +4](#)

Research across Microsoft documentation, community implementations, and enterprise architectures reveals four primary approaches: basic editor wrapper classes, separate validation result containers, advanced generic patterns, and hybrid architectural approaches. The choice depends on application complexity, with simple applications benefiting from wrapper patterns and enterprise applications requiring more sophisticated architectures with clean separation of concerns. [inspeerity](#) [Telerik](#)

Editor wrapper architectures

Basic editor wrapper pattern

The foundational pattern uses wrapper classes that encapsulate editing state alongside original values and validation messages: [Stack Overflow](#)

```
csharp
```

```
public class EditorModel<T> : INotifyPropertyChanged where T : class
{
    private T _existingValue;
    private T _newValue;
    private List<string> _validationMessages;
    private bool _isModified;

    public string Name { get; set; }

    public T ExistingValue
    {
        get => _existingValue;
        set { _existingValue = value; OnPropertyChanged(); }
    }

    public T NewValue
    {
        get => _newValue;
        set { _newValue = value; _isModified = true; OnPropertyChanged(); }
    }

    public List<string> ValidationMessages
    {
        get => _validationMessages ??= new List<string>();
        set { _validationMessages = value; OnPropertyChanged(); }
    }

    public bool IsModified => _isModified;
    public bool IsValid => ValidationMessages.Count == 0;
    public bool HasChanges => !EqualityComparer<T>.Default.Equals(ExistingValue, NewValue);

    public event PropertyChangedEventHandler PropertyChanged;

    public void ResetToOriginal()
    {
        NewValue = ExistingValue;
        ValidationMessages.Clear();
        _isModified = false;
    }

    public void AcceptChanges()
    {
        ExistingValue = NewValue;
    }
}
```

```
    _isModified = false;  
  }  
}
```

This pattern provides **essential capabilities for tracking changes, managing validation state, and enabling reset/commit operations**. The wrapper maintains both original and edited values, enabling clean rollback functionality and dirty state tracking. ([Stack Overflow](#))

Generic editor base component

For maximum reusability, advanced implementations use generic base components: ([mikesdotnetting](#))

```
csharp
```

```

public abstract class EditorBase<TModel, TEditModel> : ComponentBase, IDisposable
    where TModel : class
    where TEditModel : class, new()
{
    [Parameter] public TModel OriginalModel { get; set; }
    [Parameter] public EventCallback<TModel> OnSave { get; set; }
    [Parameter] public EventCallback OnCancel { get; set; }

    protected TEditModel EditModel { get; set; } = new();
    protected EditContext EditContext { get; set; }
    protected ValidationMessageStore MessageStore { get; set; }

    protected abstract TEditModel MapToEditModel(TModel source);
    protected abstract TModel MapFromEditModel(TEditModel source);

    protected override void OnParametersSet()
    {
        EditModel = MapToEditModel(OriginalModel);
        EditContext = new EditContext(EditModel);
        MessageStore = new ValidationMessageStore(EditContext);

        EditContext.OnValidationRequested += HandleValidation;
        EditContext.OnFieldChanged += HandleFieldChanged;
    }

    protected virtual void HandleValidation(object sender, ValidationRequestedEventArgs e)
    {
        MessageStore.Clear();
        // Custom validation logic here
    }

    protected virtual void HandleFieldChanged(object sender, FieldChangedEventArgs e)
    {
        MessageStore.Clear(e.FieldIdentifier);
        // Field-level validation if needed
    }

    public void Dispose()
    {
        EditContext.OnValidationRequested -= HandleValidation;
        EditContext.OnFieldChanged -= HandleFieldChanged;
    }
}

```

```
}  
}
```

Validation result containers with editor models

Separation of concerns pattern

The most maintainable approach separates validation concerns from editing logic using dedicated validation containers:

```
csharp  
  
public class ValidationResult  
{  
    public bool IsValid { get; set; }  
    public Dictionary<string, List<string>> Errors { get; set; } = new();  
    public List<string> GeneralErrors { get; set; } = new();  
  
    public void AddError(string field, string message)  
    {  
        if (!Errors.ContainsKey(field))  
            Errors[field] = new List<string>();  
  
        Errors[field].Add(message);  
        IsValid = false;  
    }  
}  
  
public class FormValidationResponse  
{  
    public bool Success { get; set; }  
    public Dictionary<string, List<string>> FieldErrors { get; set; } = new();  
    public List<string> GeneralErrors { get; set; } = new();  
    public string GlobalMessage { get; set; }  
}
```

Custom validation component for server integration

Microsoft's official pattern recommends custom validation components for server-side validation integration: [Microsoft Learn +3](#)

```
csharp
```

```

public class ServerValidator : ComponentBase
{
    private ValidationMessageStore messageStore;

    [CascadingParameter]
    private EditContext CurrentEditContext { get; set; }

    protected override void OnInitialized()
    {
        messageStore = new(CurrentEditContext);
        CurrentEditContext.OnValidationRequested += (s, e) =>
            messageStore?.Clear();
        CurrentEditContext.OnFieldChanged += (s, e) =>
            messageStore?.Clear(e.FieldIdentifier);
    }

    public void DisplayErrors(Dictionary<string, List<string>> errors)
    {
        foreach (var err in errors)
        {
            messageStore?.Add(CurrentEditContext.Field(err.Key), err.Value);
        }
        CurrentEditContext?.NotifyValidationStateChanged();
    }

    public void ClearErrors()
    {
        messageStore?.Clear();
        CurrentEditContext?.NotifyValidationStateChanged();
    }
}

```

State management patterns for dirty tracking

Self-contained state management without external libraries

Blazor provides robust built-in state management through EditContext and component services.

[Microsoft Learn +3](#) The most effective pattern uses a combination of scoped services and component-level state: [Robhutton +3](#)

```

public class FormStateService
{
    private readonly Dictionary<string, object> _formStates = new();
    private readonly Dictionary<string, bool> _dirtyForms = new();

    public event Action<string, bool> FormStateChanged;

    public void RegisterForm<T>(string formId, T model)
    {
        _formStates[formId] = model;
        _dirtyForms[formId] = false;
    }

    public void UpdateFormState<T>(string formId, T model, bool isDirty)
    {
        _formStates[formId] = model;
        var wasAlreadyDirty = _dirtyForms[formId];
        _dirtyForms[formId] = isDirty;

        if (wasAlreadyDirty != isDirty)
        {
            FormStateChanged?.Invoke(formId, isDirty);
        }
    }

    public T GetFormState<T>(string formId) where T : class
    {
        return _formStates.TryGetValue(formId, out var state) ? state as T : null;
    }

    public bool HasAnyDirtyForms => _dirtyForms.Any(kvp => kvp.Value);
}

```

Change tracking with EditContext

Microsoft's built-in EditContext provides sophisticated change tracking: [Microsoft Learn +6](#)

csharp

```

public class EditFormState : ComponentBase, IDisposable
{
    [CascadingParameter] public EditContext EditContext { get; set; }
    [Parameter] public EventCallback<bool> EditStateChanged { get; set; }

    private EditFieldCollection _originalValues;
    private bool _isDirty = false;

    public bool IsDirty => _isDirty;

    protected override void OnInitialized()
    {
        _originalValues = new EditFieldCollection(EditContext.Model);
        EditContext.OnFieldChanged += OnFieldChanged;
    }

    private void OnFieldChanged(object sender, FieldChangedEventArgs e)
    {
        var fieldIdentifier = e.FieldIdentifier;
        var currentValue = fieldIdentifier.Model.GetType()
            .GetProperty(fieldIdentifier.FieldName)?
            .GetValue(fieldIdentifier.Model);

        var originalValue = _originalValues.GetOriginalValue(fieldIdentifier);
        var isDirtyNow = !Equals(currentValue, originalValue);

        if (isDirtyNow != _isDirty)
        {
            _isDirty = isDirtyNow;
            EditStateChanged.InvokeAsync(_isDirty);
        }
    }

    public void Dispose() => EditContext.OnFieldChanged -= OnFieldChanged;
}

```

Nested collections and complex types

Handling complex object hierarchies

For nested collections and complex types, the research reveals sophisticated patterns using Blazor's

`ObjectGraphDataAnnotationsValidator` combined with custom collection editors: [Pragimtech +4](#)

csharp

```
public class Employee
{
    [Required]
    public string FirstName { get; set; }

    [ValidateComplexType]
    public Department Department { get; set; }

    [ValidateComplexType]
    public List<Address> Addresses { get; set; } = new List<Address>();
}

// Nested collection editor component
public class NestedCollectionEditor<TParent, TChild> : ComponentBase
{
    [Parameter] public TParent ParentModel { get; set; }
    [Parameter] public Expression<Func<TParent, ICollection<TChild>>> CollectionExpression { get; set; }
    [Parameter] public RenderFragment<TChild> ItemTemplate { get; set; }

    private ICollection<TChild> Collection => CollectionExpression.Compile()(ParentModel);

    protected void AddItem()
    {
        var newItem = Activator.CreateInstance<TChild>();
        Collection.Add(newItem);
        NotifyModelChanged();
    }

    protected void RemoveItem(TChild item)
    {
        Collection.Remove(item);
        NotifyModelChanged();
    }

    private void NotifyModelChanged() => StateHasChanged();
}
```

Complex validation for nested objects

Microsoft's `ObjectGraphDataAnnotationsValidator` enables validation across complex object graphs:

razor

```
<EditForm Model="@ParentModel">
  <ObjectGraphDataAnnotationsValidator />
  <ValidationSummary />

  <!-- Simple Properties -->
  <InputText @bind-Value="ParentModel.FirstName" />
  <ValidationMessage For="@() => ParentModel.FirstName" />

  <!-- Nested Object -->
  <fieldset>
    <legend>Department</legend>
    <InputText @bind-Value="ParentModel.Department.DepartmentName" />
    <ValidationMessage For="@() => ParentModel.Department.DepartmentName" />
  </fieldset>

  <!-- Collection Editor -->
  <NestedCollectionEditor ParentModel="@ParentModel"
    CollectionExpression="@p => p.Addresses"
    ItemTemplate="@AddressItemTemplate" />
</EditForm>
```

Server-side validation integration patterns

Server-only validation without attributes

The most flexible approach implements validation entirely on the server without relying on validation attributes: [Microsoft Learn](#)

csharp

```
[HttpPost]
public async Task<IActionResult> ValidateUser(UserModel model)
{
    var errors = new Dictionary<string, List<string>>();

    // Database-dependent validations
    if (await _userService.EmailExistsAsync(model.Email))
    {
        errors.Add(nameof(model.Email),
            new List<string> { "Email address is already in use." });
    }

    // Complex business rules
    if (model.Department == "Finance" && !model.HasFinanceAccess)
    {
        errors.Add(nameof(model.HasFinanceAccess),
            new List<string> { "Finance department requires finance access." });
    }

    if (errors.Any())
        return BadRequest(errors);

    return Ok();
}
```

Client-side processing of server validation

The client processes server validation responses using the custom validator pattern: [Stack Overflow](#)

[microsoft](#)

csharp

```

private async Task HandleSubmit()
{
    try
    {
        var response = await Http.PostAsJsonAsync("api/validate", model);

        if (response.StatusCode == HttpStatusCode.BadRequest)
        {
            var errors = await response.Content
                .ReadFromJsonAsync<Dictionary<string, List<string>>>();

            serverValidator?.DisplayErrors(errors ?? new());
        }
        else if (response.IsSuccessStatusCode)
        {
            // Handle successful validation
            await OnSave.InvokeAsync(model);
        }
    }
    catch (Exception ex)
    {
        // Handle network/system errors
        serverValidator?.DisplayErrors(new Dictionary<string, List<string>>
        {
            ["" ] = new List<string> { "A system error occurred. Please try again." }
        });
    }
}

```

Generic and reusable pattern implementation

Type-safe generic editors

The most sophisticated reusable pattern leverages Blazor's generic type parameters: [Microsoft Learn](#)

[mikesdotnetting](#)

csharp

```
@typeparam TModel
@inherits ComponentBase
```

```
<EditForm EditContext="@EditContext" OnValidSubmit="@HandleValidSubmit">
    <ServerValidator @ref="serverValidator" />
    <ValidationSummary />
```

```
@EditorTemplate(EditModel)
```

```
<div class="form-actions">
    <button type="submit" disabled="@(!IsFormValid)" class="btn btn-primary">
        @(IsSubmitting ? "Saving..." : "Save Changes")
    </button>
    <button type="button" @onclick="HandleCancel" class="btn btn-secondary">
        Cancel
    </button>
</div>
</EditForm>
```

```
@code {
    [Parameter, EditorRequired]
    public TModel Model { get; set; } = default!;

    [Parameter, EditorRequired]
    public RenderFragment<TModel> EditorTemplate { get; set; } = default!;

    [Parameter]
    public EventCallback<TModel> OnSave { get; set; }

    [Parameter]
    public string ValidationEndpoint { get; set; }

    private TModel EditModel { get; set; }
    private EditContext EditContext { get; set; }
    private ServerValidator serverValidator;
    private bool IsSubmitting { get; set; }
    private bool IsFormValid => EditContext?.Validate() ?? false;

    protected override void OnParametersSet()
    {
        // Deep clone the model for editing
        var json = JsonSerializer.Serialize(Model);
        EditModel = JsonSerializer.Deserialize<TModel>(json);
    }
}
```

```

    EditContext = new EditContext(EditModel);
}

private async Task HandleValidSubmit()
{
    IsSubmitting = true;
    try
    {
        var response = await Http.PostAsJsonAsync(ValidationEndpoint, EditModel);

        if (response.IsSuccessStatusCode)
        {
            await OnSave.InvokeAsync(EditModel);
        }
        else if (response.StatusCode == HttpStatusCode.BadRequest)
        {
            var errors = await response.Content
                .ReadFromJsonAsync<Dictionary<string, List<string>>>>();
            serverValidator?.DisplayErrors(errors ?? new());
        }
    }
    finally
    {
        IsSubmitting = false;
    }
}
}

```

Entity editor pattern for maximum reusability

The entity editor pattern provides the highest level of reusability across different model types:

csharp

```

public class EntityEditor<TEntity, TEditModel>
    where TEntity : class
    where TEditModel : class, new()
{
    private readonly Func<TEntity, TEditModel> _toEditModel;
    private readonly Func<TEditModel, TEntity> _toEntity;
    private readonly Func<TEntity, TEntity, bool> _hasChanges;

    public TEntity OriginalEntity { get; private set; }
    public TEditModel EditModel { get; private set; }
    public bool HasChanges => _hasChanges(OriginalEntity, _toEntity(EditModel));

    public EntityEditor(
        Func<TEntity, TEditModel> toEditModel,
        Func<TEditModel, TEntity> toEntity,
        Func<TEntity, TEntity, bool> hasChanges = null)
    {
        _toEditModel = toEditModel;
        _toEntity = toEntity;
        _hasChanges = hasChanges ?? DefaultChangeDetection;
    }

    public void LoadEntity(TEntity entity)
    {
        OriginalEntity = entity;
        EditModel = _toEditModel(entity);
    }

    public TEntity GetUpdatedEntity() => _toEntity(EditModel);

    private bool DefaultChangeDetection(TEntity original, TEntity updated)
    {
        var originalJson = JsonSerializer.Serialize(original);
        var updatedJson = JsonSerializer.Serialize(updated);
        return originalJson != updatedJson;
    }
}

```

Performance optimization and memory management

Advanced performance patterns

Research reveals critical performance patterns for large-scale applications: [Blazor University](#) [Microsoft Docs](#)

Memory management for complex editors:

```
csharp

public class LargeModelEditor : ComponentBase, IDisposable
{
    private Timer _debounceTimer;
    private DotNetObjectReference<LargeModelEditor> _objectRef;

    protected override void OnInitialized()
    {
        _objectRef = DotNetObjectReference.Create(this);
    }

    public void Dispose()
    {
        _debounceTimer?.Dispose();
        _objectRef?.Dispose();

        // Clear large collections
        if (LargeCollection != null)
        {
            LargeCollection.Clear();
            LargeCollection = null;
        }

        GC.SuppressFinalize(this);
    }
}
```

Debounced validation for better performance:

```
csharp
```



```
private Timer _changeDebounceTimer;

protected void HandleModelChange()
{
    _changeDebounceTimer?.Dispose();
    _changeDebounceTimer = new Timer(async _ =>
    {
        await InvokeAsync(() => OnModelChanged.InvokeAsync(EditModel));
        StateHasChanged();
    }, null, 300, Timeout.Infinite);
}
```

Complete implementation example

Here's a complete working example that demonstrates these patterns:

```
razor
```

```
@page "/customer-editor"
```

```
@typeparam TModel
```

```
<GenericEditor Model="@Customer"
```

```
    ValidationEndpoint="/api/customers/validate"
```

```
    OnSave="@HandleSave">
```

```
<EditorTemplate Context="model">
```

```
<div class="form-group">
```

```
<label>First Name</label>
```

```
<InputText @bind-Value="model.FirstName" class="form-control" />
```

```
<ValidationMessage For="@(() => model.FirstName)" />
```

```
</div>
```

```
<div class="form-group">
```

```
<label>Email</label>
```

```
<InputText @bind-Value="model.Email" class="form-control" />
```

```
<ValidationMessage For="@(() => model.Email)" />
```

```
</div>
```

```
<fieldset class="border p-3">
```

```
<legend>Addresses</legend>
```

```
@for
```

```
(
```

```
int
```

```
i
```

```
=
```

```
0
```

```
;
```

```
i
```

```
<
```

```
model
```

```
.
```

```
Addresses
```

```
.
```

```
Count
```

```
;
```

```
i
```

```
++
```

```
)
```

```
{
```

```
var
  index
=
  i
;
```

```
<
div
```

```
class
=
"
address-editor mb-3
"
>
```

```
<
div
```

```
class
=
"
form-group
"
>
```

```
<
InputText
```

```
@bind-Value
=
"
model.Addresses[index].Street
"
```

```
class
=
"
form-control
"
```

```
placeholder
```

```
=
```

```
"
```

```
Street
```

```
"
```

```
/>
```

```
<
```

```
ValidationMessage
```

```
For
```

```
=
```

```
"
```

```
@(() => model.Addresses[index].Street)
```

```
"
```

```
/>
```

```
</
```

```
div
```

```
>
```

```
<
```

```
button
```

```
type
```

```
=
```

```
"
```

```
button
```

```
"
```

```
@onclick
```

```
=
```

```
"
```

```
() => RemoveAddress(index)
```

```
"
```

```
class
```

```
=
```

```
"
```

```
btn btn-sm btn-danger
```

```
"
```

```
>  
Remove  
</  
button  
>
```

```
</  
div  
>
```

```
}
```

```
    <button type="button" @onclick="AddAddress" class="btn btn-sm btn-secondary">  
        Add Address  
    </button>  
</fieldset>  
</EditorTemplate>  
</GenericEditor>
```

```
@code {  
    private Customer Customer = new();  
  
    protected override void OnInitialized()  
    {  
        Customer = new Customer  
        {  
            FirstName = "John",  
            Email = "john@example.com",  
            Addresses = new List<Address> { new Address { Street = "123 Main St" } }  
        };  
    }  
  
    private void AddAddress() => Customer.Addresses.Add(new Address());  
  
    private void RemoveAddress(int index) => Customer.Addresses.RemoveAt(index);  
  
    private async Task HandleSave(Customer savedCustomer)  
    {  
        // Handle successful save  
        Customer = savedCustomer;  
        // Navigate or show success message
```

```
}  
}
```

Best practices and recommendations

Based on comprehensive research, the most successful implementations follow these principles:

[inspeerity](#)

Start simple, evolve complexity: Begin with basic editor wrapper patterns and evolve to more sophisticated architectures as requirements grow. Simple applications benefit from direct component state management, while enterprise applications require clean architecture patterns. [Telerik +2](#)

Leverage Blazor's built-in validation system: `EditContext` and `ValidationMessageStore` provide robust, tested validation infrastructure. [Microsoft Learn](#) Custom validation components integrate seamlessly with this system while enabling server-side validation. [Microsoft Learn +5](#)

Separate editing state from domain models: Use dedicated edit models or wrapper classes to maintain clean separation between domain logic and UI concerns. This enables better testability and maintainability. [EntityAdam](#)

Implement proper disposal patterns: Complex editors must properly dispose of timers, event handlers, and large collections to prevent memory leaks in long-running applications. [microsoft](#) [Microsoft Docs](#)

Design for reusability from the beginning: Generic component patterns and entity editor architectures provide maximum reusability across different model types while maintaining type safety. [Blazor School](#)

[mikesdotnetting](#)

The patterns documented here provide a comprehensive foundation for building sophisticated Blazor WebAssembly editing applications that are maintainable, scalable, and provide excellent user experiences.

[github](#) [GitHub](#) Success comes from choosing the appropriate architectural complexity for your specific requirements and evolving the implementation as needs grow. [inspeerity](#) [Telerik](#)